REPORT DOCUMENT

- ## TASK 1:

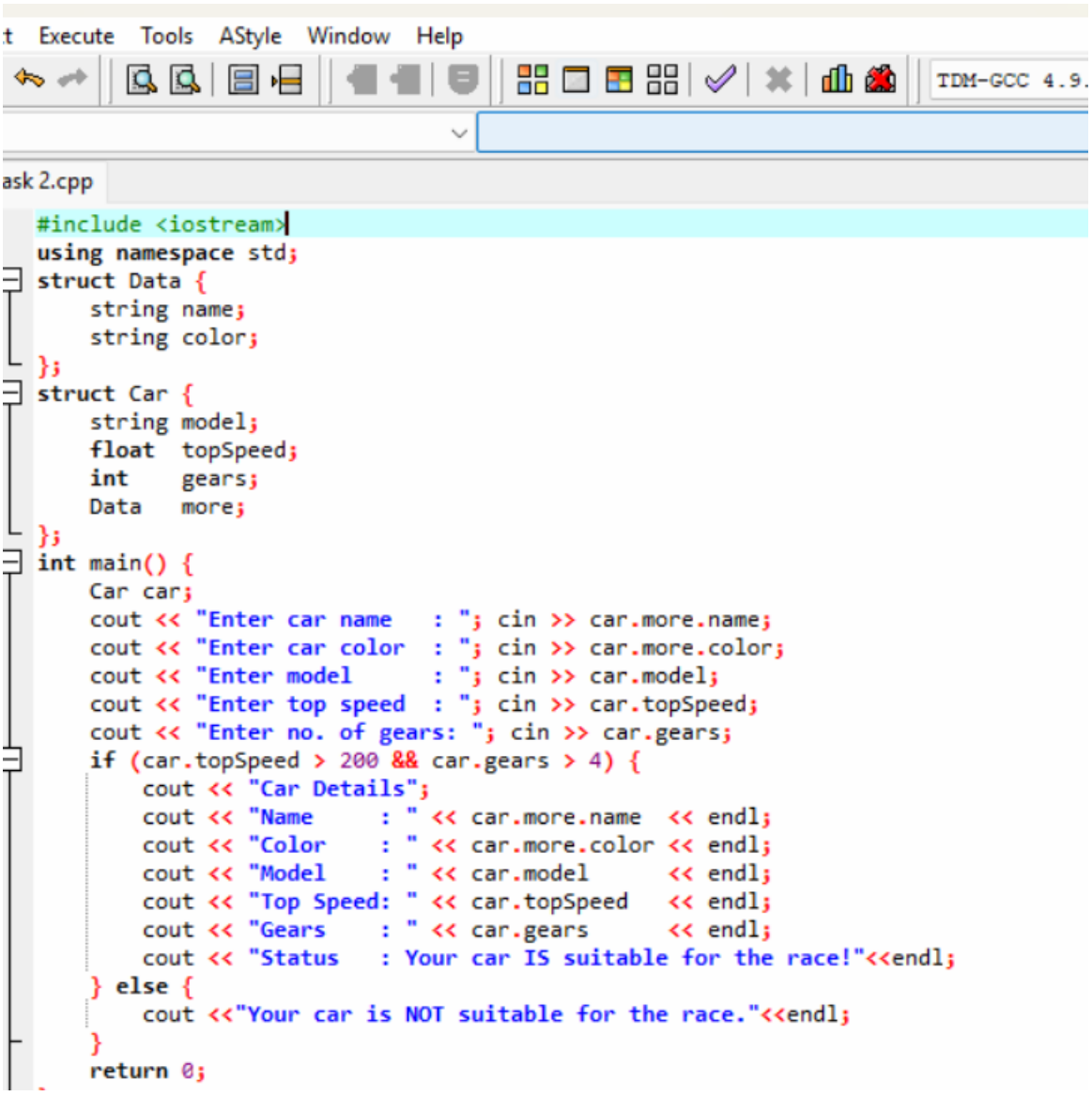
task 1.cpp

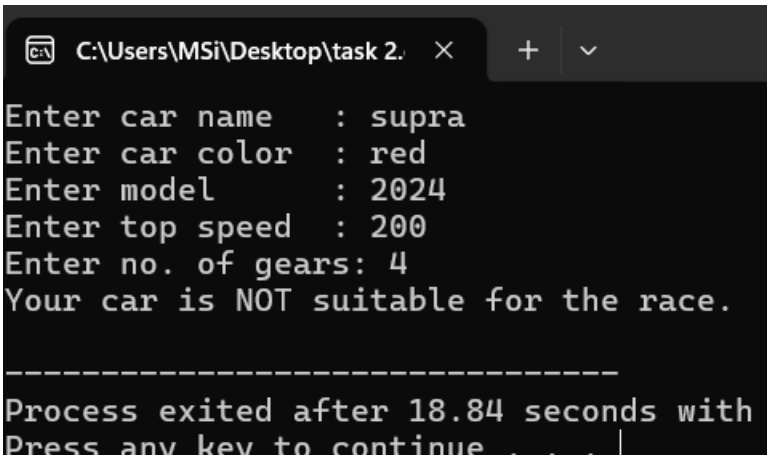
```cpp
#include <iostream>
using namespace std;
struct Result {
    float marks;
    char  grade;
};
struct Record {
    int    rollNo;
    Result result;
};
int main() {
    Record student;
    cout << "Enter roll number: ";
    cin  >> student.rollNo;
    cout << "Enter marks: ";
    cin  >> student.result.marks;
    cout << "Enter grade";
    cin  >> student.result.grade;
    cout << "Student Record"<<endl;
    cout << "Roll Number : " << student.rollNo<<endl;
    cout << "Marks       : " << student.result.marks<<endl;
    cout << "Grade       : " << student.result.grade<<endl;
    return 0;
}
```

```
C:\Users\MSi\Desktop\task 1.
Enter roll number: 3
Enter marks: 78
Enter gradeb
Student Record
Roll Number : 3
Marks       : 78
Grade       : b

--------------------------
```

- ## TASK 2:


t Execute Tools AStyle Window Help
TDM-GCC 4.9.
ask 2.cpp
#include <iostream>
using namespace std;
struct Data {
string name;
string color;
};
struct Car {
string model;
float topSpeed;
int gears;
Data more;
};
int main() {
Car car;
cout << "Enter car name : "; cin >> car.more.name;
cout << "Enter car color : "; cin >> car.more.color;
cout << "Enter model : "; cin >> car.model;
cout << "Enter top speed : "; cin >> car.topSpeed;
cout << "Enter no. of gears: "; cin >> car.gears;
if (car.topSpeed > 200 && car.gears > 4) {
cout << "Car Details";
cout << "Name : " << car.more.name << endl;
cout << "Color : " << car.more.color << endl;
cout << "Model : " << car.model << endl;
cout << "Top Speed: " << car.topSpeed << endl;
cout << "Gears : " << car.gears << endl;
cout << "Status : Your car IS suitable for the race!"<<endl;
} else {
cout <<"Your car is NOT suitable for the race."<<endl;
}
return 0;


C:\Users\MSi\Desktop\task 2.
Enter car name : supra
Enter car color : red
Enter model : 2024
Enter top speed : 200
Enter no. of gears: 4
Your car is NOT suitable for the race.
Process exited after 18.84 seconds with
Press any key to continue

- TASK 3:

task 3 new one.cpp

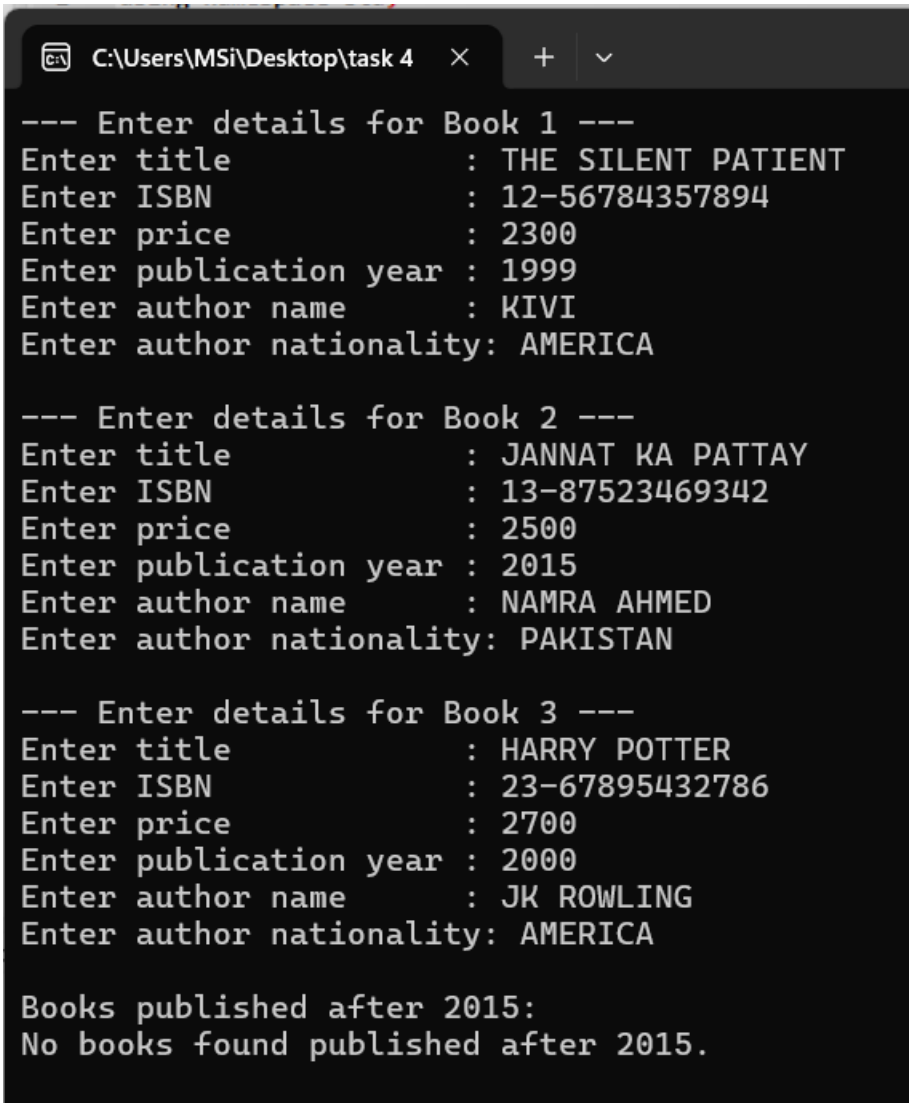
```cpp
struct Complex {
    float real;
    float imag;
};
Complex addition(Complex a, Complex b) {
    Complex res;
    res.real = a.real + b.real;
    res.imag = a.imag + b.imag;
    return res;
}
Complex subtract(Complex a, Complex b) {
    Complex res;
    res.real = a.real - b.real;
    res.imag = a.imag - b.imag;
    return res;
}
Complex multiply(Complex a, Complex b) {
    Complex res;
    res.real = a.real * b.real - a.imag * b.imag;
    res.imag = a.real * b.imag + a.imag * b.real;
    return res;
}
void print(const char* label, Complex c) {
    cout << label << c.real;
    if (c.imag >= 0) cout << " + " <<  c.imag << "i"<<endl;
    else             cout << " - " << -c.imag << "i"<<endl;
}
int main() {
    Complex c1, c2;
    cout << "Enter first complex number  (real+imag): ";
    cin  >> c1.real >> c1.imag;
    cout << "Enter second complex number (real+imag): ";
    cin  >> c2.real >> c2.imag;
    Complex sum  = addition(c1, c2);
    Complex diff = subtract(c1, c2);
    Complex prod = multiply(c1, c2);
    cout << "Results"<<endl;
    print("Addition    : ", sum);
    print("Subtraction : ", diff);
    print("Multiply    : ", prod);
    return 0;
}
```

```
C:\Users\MSi\Desktop\task 3
Enter first complex number  (real+imag): 2 -4
Enter second complex number (real+imag): 3 -5
Results
Addition    : 5 - 9i
Subtraction : -1 + 1i
Multiply    : -14 - 22i
```

- TASK 4:

task 4 new.cpp

```cpp
#include <iostream>
#include<string>
using namespace std;
struct Author {
    string name;
    string nationality;
};
struct Book {
    string title;
    string ISBN;
    double price;
    int    pubYear;
    Author author;
};
int main() {
    Book library[3];
    for (int i = 0; i < 3; i++) {
        cout << " Enter details for Book " << i + 1 << " ---" << endl;
        cout << "Enter title      : ";
        cin  >> library[i].title;
        cout << "Enter ISBN       : ";
        cin  >> library[i].ISBN;
        cout << "Enter price      : ";
        cin  >> library[i].price;
        cout << "Enter publication year   : ";
        cin  >> library[i].pubYear;
        cout << "Enter author name        : ";
        cin  >> library[i].author.name;
        cout << "Enter author nationality : ";
        cin  >> library[i].author.nationality;
    }
    cout << "Books published after 2015:" << endl;
```



Q1. What is the difference between accessing a member of a simple struct vs. a nested struct? Give dot-notation examples from this task.

Answer:

- FOR SIMPLE STRUCT:

You can use one dot to reach it directly

E:g: library[i].title

library[i].price

- FOR NESTED STRUCT:

First to reach the inner struct and then inside the member of it
E:g: library[i].author.name
library[i].author.nationality

Q2. Why is it better to store the Author as a nested struct inside Book rather than having 'authorName' and 'authorNationality' as direct string members of Book?

Answer:

name and nationality are properties that belong to an author, not to a book but if you later need to add more author details (e.g., email, birth year, country code), you only update
the Author struct in one place. With direct fields
like authorName you would need to modify every struct that uses them.

Q3. If you had to add a 'co-author' to the Book struct, how would you modify the struct definition?
Write just the updated Book struct.

Answer:

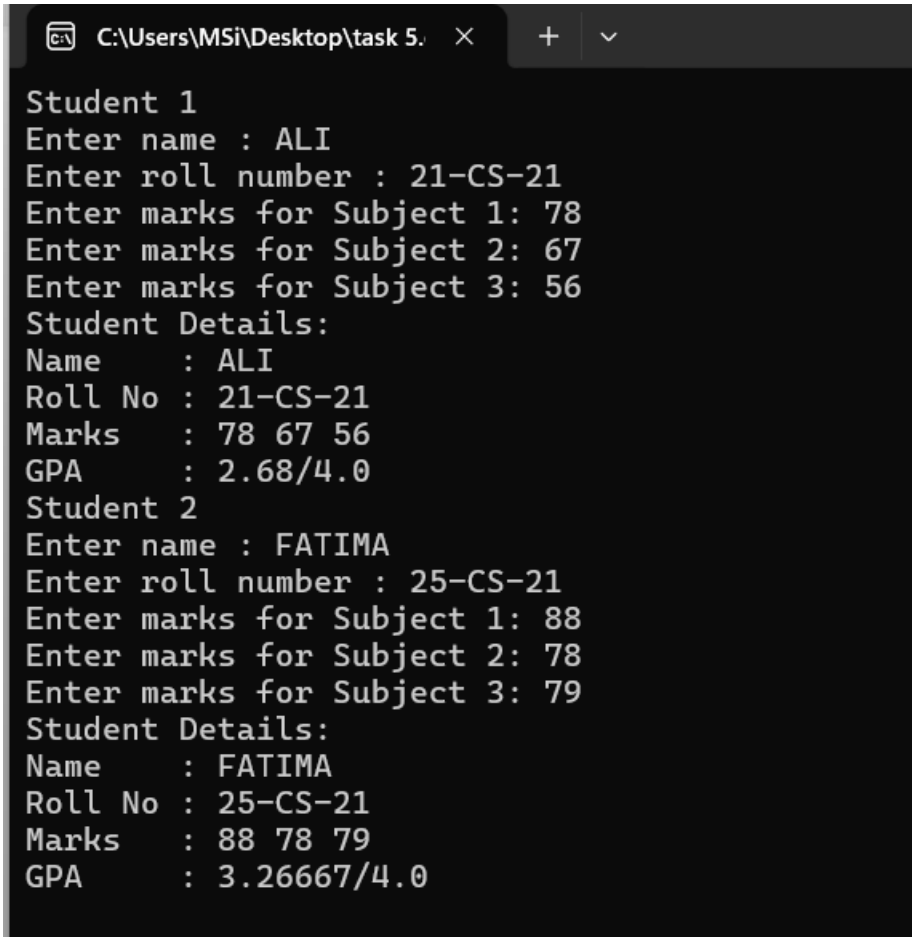
```
struct Book {
    string title;
    string ISBN;
    double price;
```

int pubYear;
Author author;
Author coAuthor;
};
Because Author is already a reusable struct, adding a co-author costs exactly one line will access it
as library[i].coAuthor.name and library[i].coAuthor.nationality.

- TASK 5:

task 5.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;
struct Student {
    string name;
    string rollNo;
    float  marks[3];
    float  gpa;
};
void calculateGPA(Student &s) {
    s.gpa = (s.marks[0] + s.marks[1] + s.marks[2])/300.0*4.0;
}
void displayStudent(Student s) {
    cout << "Student Details:"<<endl;
    cout << "Name    : " << s.name    << endl;
    cout << "Roll No : " << s.rollNo  << endl;
    cout << "Marks   : " << s.marks[0] <<" "
                         << s.marks[1] <<" "
                        << s.marks[2] << " "<<endl;
    cout << "GPA     : " << s.gpa    <<  "/4.0"<<endl;
}
int main() {
    Student students[2];
    for (int i = 0; i < 2; i++) {
        cout << "Student " << i + 1 <<endl;
        cout << "Enter name : ";
        cin  >> students[i].name;
        cout << "Enter roll number : ";
        cin  >> students[i].rollNo;
        for (int j = 0; j < 3; j++) {
            cout << "Enter marks for Subject " << j + 1 << ": ";
            cin  >> students[i].marks[j];
```



Q1. What is the key difference between passing a struct by value vs. by reference? In this task, why must calculateGPA() use pass-by-reference?

Answer:

Pass by value: makes a full copy of the struct. Any changes inside the function are lost when it returns — the original in main() is untouched.

Pass-by-reference: gives the function direct access to the

original struct in memory. Changes persist after the function returns.
calculateGPA() must use pass-by-reference because it writes the computed GPA back into s.gpa. If it used pass-by-value, the GPA would be calculated on a copy and thrown away So, the Student in main() would always have gpa = 0.

Q2

If displayStudent() accidentally modifies s.gpa inside the function (pass-by-value), will it affect the original struct in main()? Why or why not?

ANSWER:

No, it will **not** affect the original. Pass-by-value creates a completely independent copy of the Student struct.
Any modification to s.gpa inside displayStudent() only changes that local copy, which is destroyed when the function ends. The original Student object in main() remains unchanged. This is exactly why pass-by-value is safe for display functions.

Q3. Can a function return a struct in C++? Rewrite the signature of calculateGPA() so it returns a float instead of modifying the struct by reference.

Answer:

Yes, a function can return a struct in C++. The rewritten signature would be:

```
float calculateGPA(Student s) {
    float gpa = (s.marks[0] + s.marks[1] + s.marks[2]) / 300.0 * 4.0;
    if (gpa > 4.0) gpa = 4.0;
    return gpa;
}
```

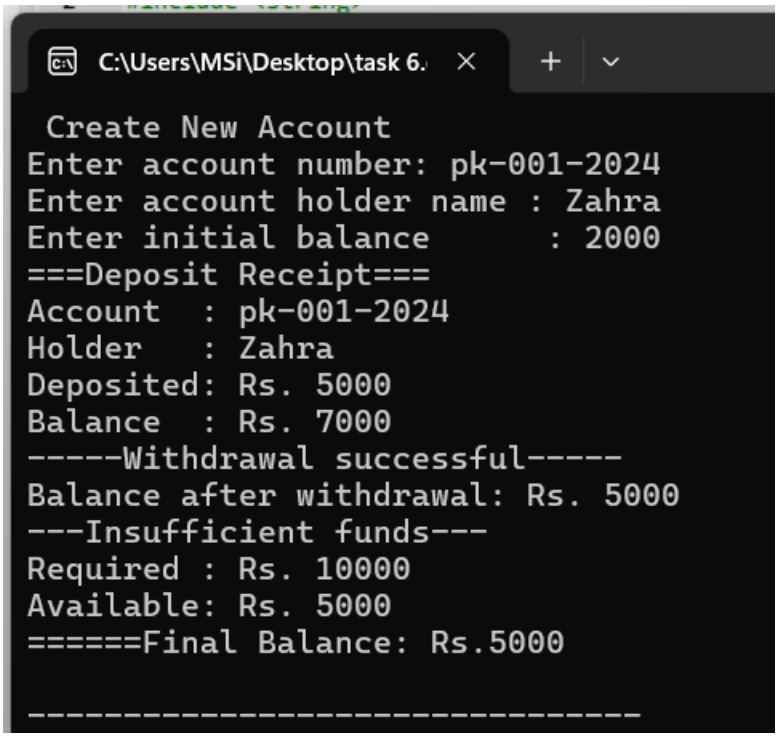
The caller must then manually assign the returned value.

```
students[i].gpa = calculateGPA(students[i]);
```

- TASK 6:

task 6.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;
struct Account {
    string accountNumber;
    string holderName;
    double balance;
};
Account createAccount() {
    Account acc;
    cout << " Create New Account"<<endl;
    cout << "Enter account number: "; cin >> acc.accountNumber;
    cout << "Enter account holder name : "; cin >> acc.holderName;
    cout << "Enter initial balance      : "; cin >> acc.balance;
    return acc;
}
void deposit(Account &acc, double amount) {
    acc.balance += amount;
    cout << "===Deposit Receipt==="<<endl;
    cout << "Account  : " << acc.accountNumber << endl;
    cout << "Holder   : " << acc.holderName    << endl;
    cout << "Deposited: Rs. " << amount   << endl;
    cout << "Balance  : Rs. " << acc.balance      << endl;
}
bool withdraw(Account &acc, double amount) {
    if (amount > acc.balance) {
        cout << "---Insufficient funds---"<<endl;
        cout << "Required : Rs. " << amount      << endl;
        cout << "Available: Rs. " << acc.balance  << endl;
        return false;
    }
    acc.balance -= amount;
```



Q1. What does it mean for a function to 'return a struct'? How is createAccount() in this task different from the display/calculate functions in Lab Task 5?

Answer:

Returning a struct means the function creates a struct locally and hands a copy of it back to the caller. The caller can then store it in a variable: Account myAcc = createAccount();

Difference :

— displayStudent() takes a struct as input and returns nothing (void).

— calculateGPA() takes a struct as input (by reference) and modifies it.

— createAccount() takes no input at all and **produces a** new struct as its output.

Q2. In the withdraw() function, why is bool a better return type than void for this scenario?

Answer:

With a void return type, the caller (main) has no way to know whether the withdrawal actually succeeded or failed.
Returning bool gives the caller "**decision-making power"**. In main(), you can write:

```
if (withdraw(myAcc, 2000))
    cout << "Proceed with transaction.";
else
    cout << "Alert the user to retry this.";
```

Q3. If Account had a nested struct (e.g., a BranchInfo struct inside Account), how would you access the branch name inside the deposit() function? Write the access expression.

Answer:

```
struct BranchInfo { string branchName; string city; };
struct Account {
    string     accountNumber;
    string     holderName;
    double     balance;
    BranchInfo branch;
```

};

Inside deposit(), access branch name like this:

"acc.branch.branchName"

- HOME TASK 1:

[*] home task 1.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;
struct Date {
    int day;
    int month;
    int year;
};
struct Phonebook {
    string name;
    string city;
    string phone;
    Date   date;
};
int main() {
    Phonebook pb;
    cout << "Enter name: "; cin >> pb.name;
    cout << "Enter city : "; cin >> pb.city;
    cout << "Enter phone number: "; cin >> pb.phone;
    cout << "Enter day : "; cin >> pb.date.day;
    cout << "Enter month : "; cin >> pb.date.month;
    cout << "Enter year : "; cin >> pb.date.year;
    cout << "\n--- Phonebook Entry ---\n";
    cout << "Name  : " << pb.name  <<endl;
    cout << "City  : " << pb.city  <<endl;
    cout << "Phone : " << pb.phone <<endl;
    cout << "Date  : " << pb.date.day   << "/"<< pb.date.month << "/"
                       << pb.date.year  <<endl;
    return 0;
}
```

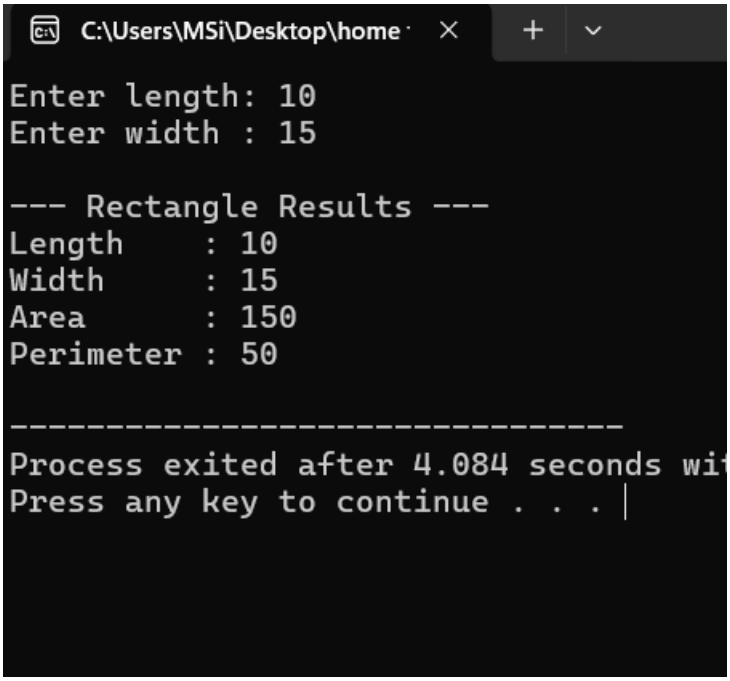
```
C:\Users\MSi\Desktop\home
Enter name: Tawasal
Enter city : lhr
Enter phone number: 789906544
Enter day : monday
Enter month : Enter year :
--- Phonebook Entry ---
Name  : Tawasal
City  : lhr
Phone : 789906544
Date  : 0/0/1

--------------------------------
```

- HOME TASK 2:

home task 2.cpp

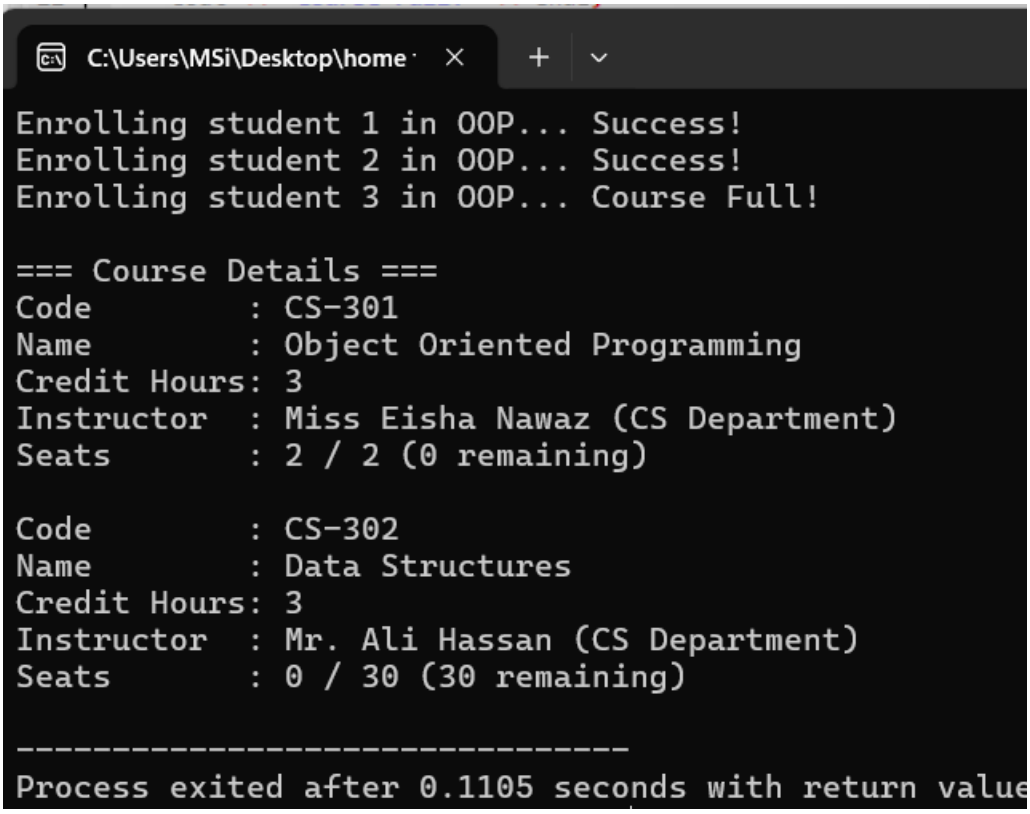
```cpp
#include <iostream>
using namespace std;
struct Parameters {
    double length;
    double width;
};
struct Result {
    double area;
    double perimeter;
};
struct Rectangle {
    Parameters info;
    Result     result;
};
int main() {
    Rectangle rect;
    cout << "Enter length: "; cin >> rect.info.length;
    cout << "Enter width : "; cin >> rect.info.width;
    rect.result.area      = rect.info.length * rect.info.width;
    rect.result.perimeter = 2 * (rect.info.length + rect.info.width);
    cout << "\n--- Rectangle Results ---\n";
    cout << "Length    : " << rect.info.length    << endl;
    cout << "Width     : " << rect.info.width     << endl;
    cout << "Area      : " << rect.result.area      << endl;
    cout << "Perimeter : " << rect.result.perimeter << endl;
    return 0;
}
```



- HOME TASK 3:

[*] home task 3.cpp

```cpp
#include <string>
using namespace std;
struct Instructor {
    string name;
    string department;
};
struct Course {
    string     courseCode;
    string     courseName;
    int        creditHours;
    int        maxSeats;
    int        enrolledStudents;
    Instructor instructor;
};
bool enrollStudent(Course &c) {
    if (c.enrolledStudents < c.maxSeats) {
        c.enrolledStudents++;
        return true;
    }
    cout << "Course Full!" << endl;
    return false;
}
void displayCourse(Course c) {
    int remaining = c.maxSeats - c.enrolledStudents;
    cout << "Code        : " << c.courseCode  <<endl;
    cout << "Name        : " << c.courseName  << endl;
    cout << "Credit Hours: " << c.creditHours << endl;
    cout << "Instructor  : " << c.instructor.name
         << " (" << c.instructor.department << ")\n";
    cout << "Seats       : " << c.enrolledStudents << " / " << c.maxSeats << " (" << remaining << " remaining)\n";
}
```



Q1. What happens to the original Course object if displayCourse() is called with pass-by-value and modifies enrolledStudents inside? Would it change in main()?

Answer:

No, it would **not** change in main(). Pass-by-value creates a complete independent copy of the Course object. Any changes made inside displayCourse() only affect that local copy, which is destroyed when the function returns. The original Course in main() stays exactly as it was. This is why pass-by-value is the safe choice for display(only) functions.

Q2. Explain why enrollStudent() must use pass-by-reference to work correctly in this system.

Answer:

enrollStudent() needs to permanently increment enrolledStudents in the actual Course object. If pass-by-value were used, the increment would happen on a temporary copy and be thrown away.The counter in main() would stay at 0 forever. Pass-by-reference makes the function work directly on the original object in memory, so the change persists after the function returns.

Q3. How would you extend this system to store an array of 5 Courses? Write only the declaration line in main().

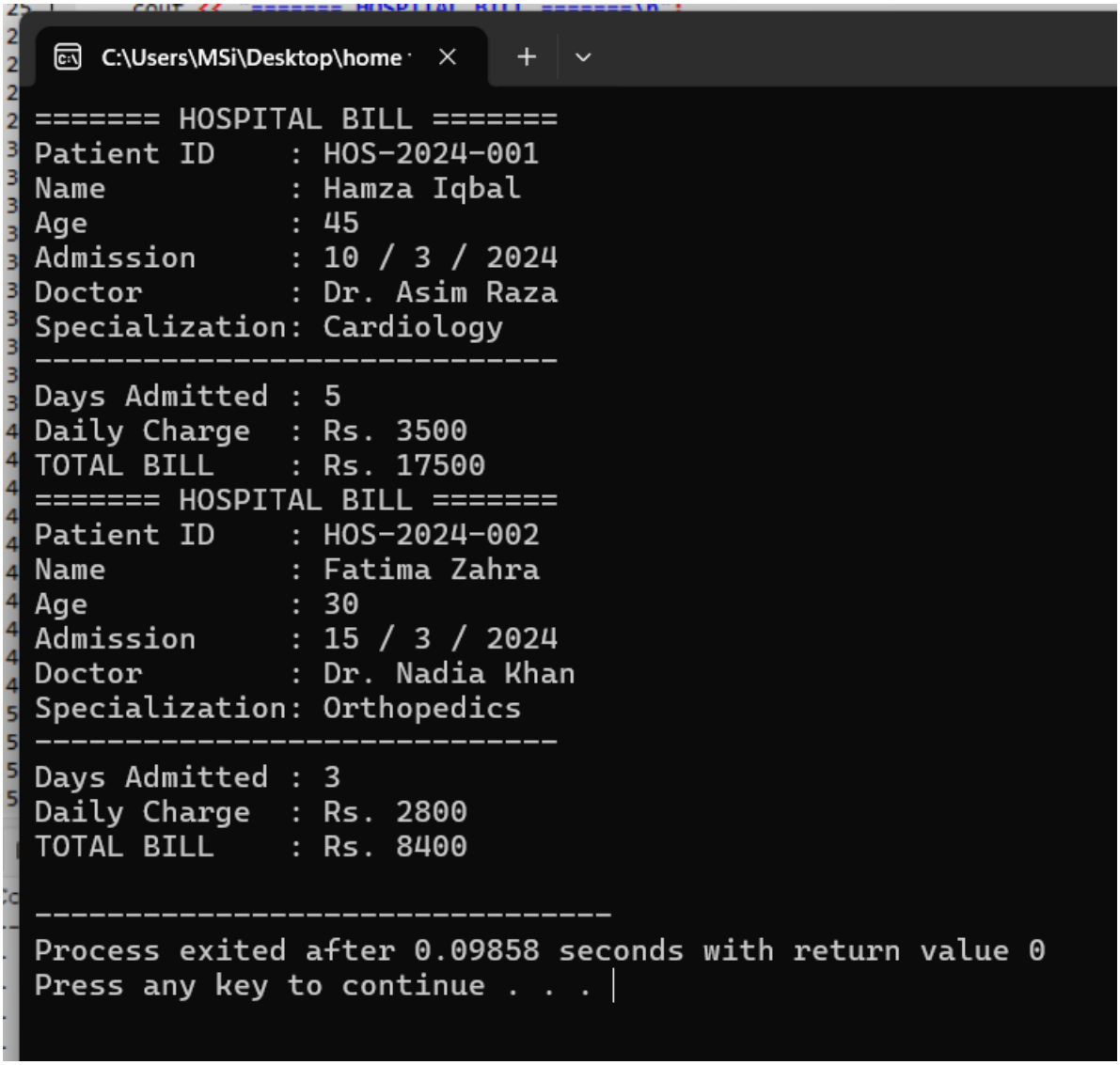
Answer:

Course courses[5];
Access individual elements as courses[0], courses[1], etc

- HOME TASK 4:

[*] home task 4.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;
struct Date {
    int day;
    int month;
    int year;
};
struct Doctor {
    string name;
    string specialization;
};
struct Patient {
    string patientID;
    string name;
    int    age;
    Date   admissionDate;
    Doctor assignedDoctor;
    double dailyCharge;
};
double calculateBill(Patient p, int days) {
    return p.dailyCharge * days;
}
void displayPatientReport(Patient p, int days) {
    cout << "======= HOSPITAL BILL ======="<<endl;
    cout << "Patient ID    : " << p.patientID                  << endl;
    cout << "Name          : " << p.name                        << endl;
    cout << "Age           : " << p.age                         << endl;
    cout << "Admission     : " << p.admissionDate.day    << " / " << p.admissionDate.month  << " / " << p.admissionDate.year   << endl;
    cout << "Doctor        : " << p.assignedDoctor.name         << endl;
    cout << "Specialization: " << p.assignedDoctor.specialization << endl;
    cout << "----------------------------\n";
    cout << "Days Admitted : " << days                          << endl;
    cout << "Daily Charge  : Rs. " << p.dailyCharge             << endl;
    cout << "TOTAL BILL    : Rs. " << calculateBill(p, days)    << endl;
}
int main() {
    Patient p1;
    p1.patientID                    = "HOS-2024-001";
    p1.name                         = "Hamza Iqbal";
    p1.age                          = 45;
```



Q1. This task uses THREE levels of nesting (Patient -Date, Patient - Doctor). How do you access the doctor specialization from a Patient variable p? Write the exact expression.

Answer:

“p.assignedDoctor.specialization”

This way we can get access to doctor specialization from patient variable P.

Q2. calculateBill() takes Patient by value (not reference). Would the function still work if you changed it to pass-by-reference? What would be the practical difference in this specific case?

Answer:

Yes, it would still work correctly either way since the function only **reads** data — it never modifies the Patient. The practical difference is performance: pass-by-value makes a full copy of the entire Patient struct on every call (slightly more memory and time), while pass-by-reference avoids the copy entirely.
"double calculateBill(const Patient &p, int days)"

Q3. If you wanted to store admission history as an array of 5 Date objects inside Patient, how would you modify the Patient struct? Write only the modified struct.

Answer:

```
struct Patient {
    string patientID;
    string name;
    int    age;
    Date   admissionHistory[5];
    Doctor assignedDoctor;
    double dailyCharge;
```

ग़